```
# Pertemuan 2: Convolutional Neural Network (Fashion-MNIST)

# %% Setup
import os
from pathlib import Path

# Simpan cache dataset di folder proyek, sebelum mengimpor TensorFlow.
os.environ["KERAS_HOME"] = str(Path.cwd() / ".tools" / "keras")

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

tf.keras.utils.set_random_seed(42)
print("TensorFlow:", tf.__version__)

EPOCHS = 5
BATCH_SIZE = 128


# %% Cross-correlation
gambar = tf.constant([[[[1.], [2.], [3.]],
                       [[4.], [5.], [6.]],
                       [[7.], [8.], [9.]]]])

kernel = tf.reshape(
    tf.constant([1., 0., 0., -1.]),
    (2, 2, 1, 1)
)

hasil = tf.nn.conv2d(
    gambar,
    kernel,
    strides=1,
    padding="VALID"
)

print("Input:\n", gambar.numpy()[0, :, :, 0])
print("Cross-correlation:\n", hasil.numpy()[0, :, :, 0])

assert np.allclose(hasil.numpy(), -4)


# %% Pengaruh padding dan stride
contoh = tf.zeros((1, 28, 28, 1))

for padding, stride in [("valid", 1), ("same", 1), ("same", 2)]:
    layer = tf.keras.layers.Conv2D(
        4,
        3,
        padding=padding,
        strides=stride
    )
    print(
        f"padding={padding}, stride={stride}:",
        layer(contoh).shape
    )


# %% Input multi-channel (RGB)
contoh_rgb = tf.zeros((1, 28, 28, 3))

conv_rgb = tf.keras.layers.Conv2D(
    8,
    3,
    padding="same"
)

print(
    "Input RGB:",
    contoh_rgb.shape,
    "Output:",
    conv_rgb(contoh_rgb).shape
)
print(
    "Parameter:",
    conv_rgb.count_params(),
    "= (3 * 3 * 3 + 1) * 8"
)
```

```
)


# %% Pooling
nilai = tf.reshape(
    tf.range(1, 17, dtype=tf.float32),
    (1, 4, 4, 1)
)

print("Input:\n", nilai.numpy()[0, :, :, 0])
print(
    "Max pooling:\n",
    tf.keras.layers.MaxPooling2D(2)(nilai).numpy()[0, :, :, 0]
)
print(
    "Average pooling:\n",
    tf.keras.layers.AveragePooling2D(2)(nilai).numpy()[0, :, :, 0]
)


# %% Memuat dataset Fashion-MNIST
(x_pool, y_pool), (x_test, y_test) = \
    tf.keras.datasets.fashion_mnist.load_data()

nama_kelas = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

x_train, x_val, y_train, y_val = train_test_split(
    x_pool,
    y_pool,
    train_size=12000,
    test_size=3000,
    stratify=y_pool,
    random_state=42
)

x_train = x_train.astype("float32")[..., np.newaxis] / 255.0
x_val = x_val.astype("float32")[..., np.newaxis] / 255.0
x_test = x_test.astype("float32")[..., np.newaxis] / 255.0

print(
    "Train:",
    x_train.shape,
    "Validation:",
    x_val.shape,
    "Test:",
    x_test.shape
)


# %% Contoh gambar dari setiap kelas
fig, axes = plt.subplots(2, 5, figsize=(10, 4))

for kelas, ax in enumerate(axes.flat):
    idx = np.flatnonzero(y_train == kelas)[0]
    ax.imshow(
        x_train[idx, :, :, 0],
        cmap="gray",
        vmin=0,
        vmax=1
    )
    ax.set_title(
        f"{kelas}: {nama_kelas[kelas]}",
        fontsize=9
    )
    ax.axis("off")

plt.tight_layout()
plt.show()


# %% Membangun model CNN
tf.keras.utils.set_random_seed(42)

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),

    tf.keras.layers.Conv2D(
        8,
        3,
        padding="same",
```

```
        activation="relu",
        name="fitur_awal"
    ),

    tf.keras.layers.MaxPooling2D(2),

    tf.keras.layers.Conv2D(
        16,
        3,
        padding="same",
        activation="relu"
    ),

    tf.keras.layers.MaxPooling2D(2),

    tf.keras.layers.Flatten(),

    tf.keras.layers.Dense(32, activation="relu"),

    tf.keras.layers.Dense(10, activation="softmax")
], name="cnn_sederhana")

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()


# %% Pelatihan
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=2
)


# %% Grafik loss dan akurasi
def plot_history(history, judul):
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
    epochs = range(1, len(history.history["loss"]) + 1)

    for ax, metric in zip(axes, ["loss", "accuracy"]):
        ax.plot(
            epochs,
            history.history[metric],
            label="Train"
        )
        ax.plot(
            epochs,
            history.history["val_" + metric],
            label="Validation"
        )
        ax.set(
            xlabel="Epoch",
            ylabel=metric,
            title=metric
        )
        ax.legend()

    fig.suptitle(judul)
    plt.tight_layout()
    plt.show()


plot_history(history, "CNN sederhana")


# %% Visualisasi feature map
ekstraktor = tf.keras.Model(
    inputs=model.inputs,
    outputs=model.get_layer("fitur_awal").output
)

fitur = ekstraktor(
    x_val[:1],
    training=False
).numpy()[0]
```

```
fig, axes = plt.subplots(1, 9, figsize=(15, 2))

axes[0].imshow(x_val[0, :, :, 0], cmap="gray")
axes[0].set_title("Input")

for i in range(8):
    axes[i + 1].imshow(fitur[:, :, i], cmap="viridis")
    axes[i + 1].set_title(f"Fitur {i + 1}")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()


# %% Evaluasi pada data uji
test_loss, test_acc = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(
    f"Test loss: {test_loss:.4f} | "
    f"Test accuracy: {test_acc:.2%}"
)

prob = model.predict(x_test[:10], verbose=0)
pred = prob.argmax(axis=1)

fig, axes = plt.subplots(2, 5, figsize=(12, 5))

for i, ax in enumerate(axes.flat):
    ax.imshow(x_test[i, :, :, 0], cmap="gray")
    ax.set_title(
        f"Asli: {nama_kelas[y_test[i]]}\n"
        f"Pred: {nama_kelas[pred[i]]}",
        color="green" if pred[i] == y_test[i] else "red",
        fontsize=9
    )
    ax.axis("off")

plt.tight_layout()
plt.show()
```

```
TensorFlow: 2.21.0
Input:
 [[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
Cross-correlation:
 [[-4. -4.]
 [-4. -4.]]
padding=valid, stride=1: (1, 26, 26, 4)
padding=same, stride=1: (1, 28, 28, 4)
padding=same, stride=2: (1, 14, 14, 4)
Input RGB: (1, 28, 28, 3) Output: (1, 28, 28, 8)
Parameter: 224 = (3 * 3 * 3 + 1) * 8
Input:
 [[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]
 [13. 14. 15. 16.]]
Max pooling:
 [[ 6.  8.]
 [14. 16.]]
Average pooling:
 [[ 3.5  5.5]
 [11.5 13.5]]
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Train: (12000, 28, 28, 1) Validation: (3000, 28, 28, 1) Test: (10000, 28, 28, 1)
```

